

APB BASED SPI DESIGN

Project submitted by: Kotra Hamsika

Course: VLSI Design Internship

Batch: B2B DI 08

INDEX:

S. No	Content	Page No.
1	Project Overview	1
2	Architecture	2
3	Top Level Block Diagram	3
4	APB Slave Interface	4-5
4.1	Block Diagram	
4.2	Simulation Waveform	
4.3	Synthesis Result	
5	Baud Rate Generator	6-8
5.1	Block Diagram	
5.2	Simulation Waveform	
5.3	Synthesis Result	
6	SPI Slave Control Select	9-11
6.1	Block Diagram	
6.2	Simulation Waveform	
6.3	Synthesis Result	

S. No	Content	Page No.
7	Shift Register	12-13
7.1	Block Diagram	
7.2	Simulation Waveform	
7.3	SynthesisResult	
8	SPI Core Wave Forms	14
9	TopModuleSynthesisResult	15
10	Conclusion	16

1. Project Overview:

The **APB-Based SPI** design is implemented in **Verilog HDL** and facilitates seamless communication between an **Advanced Peripheral Bus (APB)** and **Serial Peripheral Interface (SPI)** devices. It serves as a dependable and high-performance interface for data exchange between a processor and external peripherals, including **sensors, memory devices, ADCs, and DACs**.

The architecture comprises four key functional blocks: the **APB Slave Interface, Baud Rate Generator, SPI Slave Control Select, and Shift Register**. Each of these modules plays a distinct role, collectively handling **APB protocol transactions, SPI clock generation, slave device selection, and serial data transmission and reception**.

The **SPI Master** is designed to support **full-duplex data transfer, configurable baud rate generation, and multiple SPI operating modes (CPOL/CPHA)**, making it well-suited for a broad spectrum of **embedded systems and SoC applications**. Its **modular design** approach not only streamlines the processes of design and verification but also facilitates future scalability and enhancements, ensuring robust and efficient communication across all supported peripherals.

2.Architecture:

The top-level architecture of the **APB-Based SPI Master Core** integrates both **APB protocol** and **SPI capabilities** into a unified design. It consists of four distinct modules that work in coordination to manage **register access, clock generation, control logic, and data shifting**.

The primary inputs to the system include the **APB bus signals** — **PCLK, PRESETn, PADDR, PWRITE, PSEL, PENABLE, and PWDATA** — along with the **MISO line** from the **SPI bus**. The outputs encompass **APB response signals** such as **PRDATA, PREADY, and PSLVERR**, as well as **SPI bus signals** including **SS, SCLK, and MOSI**, and a dedicated **interrupt request line**.

The architecture is built around four internal functional blocks :

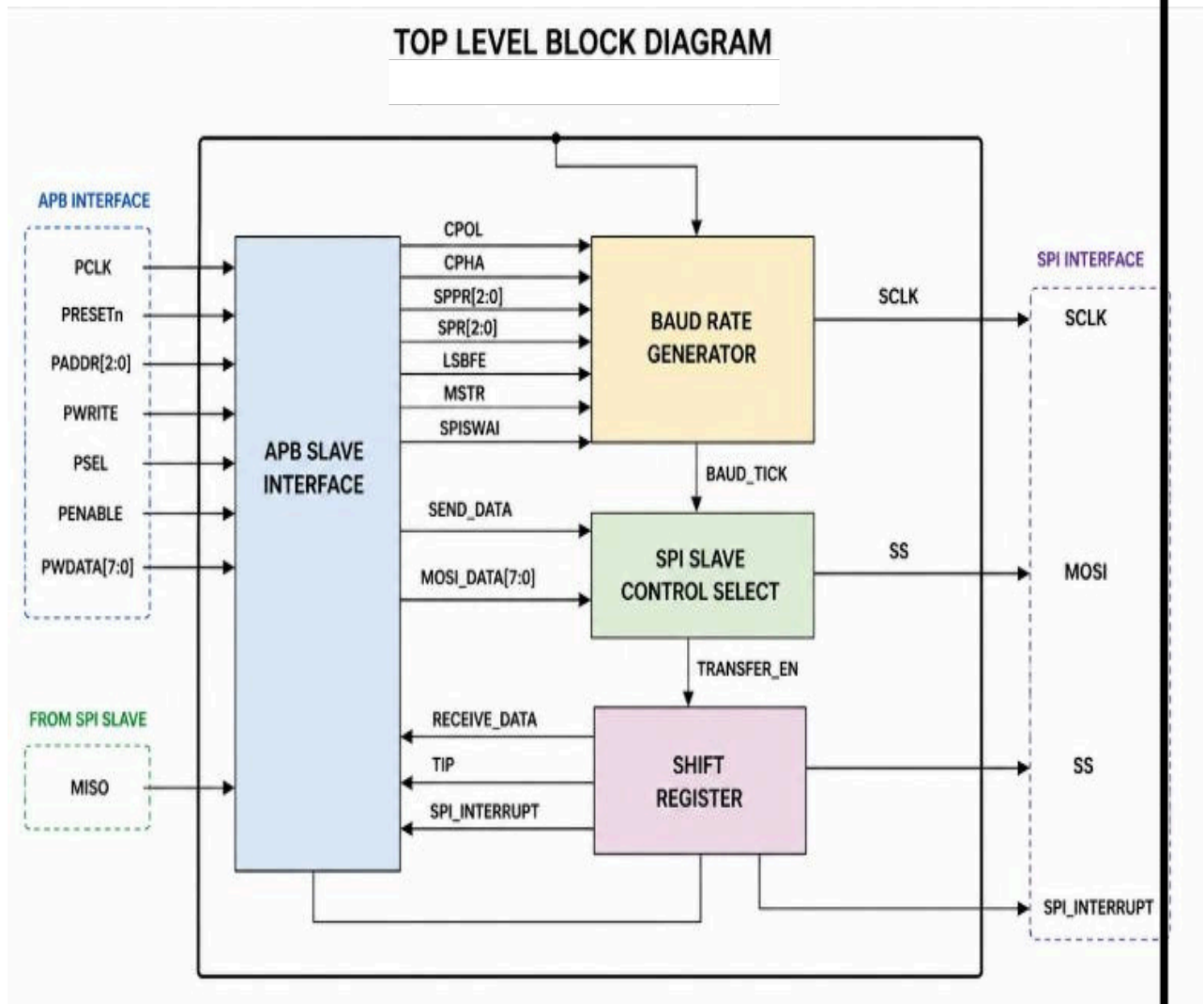
APB Slave Interface: Acts as the bridge between the system's **APB bus** and the **SPI core's internal registers**, enabling seamless register read and write operations.

Baud Rate Generator: Responsible for producing the **serial clock (SCLK)** and its associated timing signals, ensuring accurate and configurable clock generation for SPI communication.

SPI Slave Control Select: Oversees the **high-level state management** of SPI operations, controlling slave selection and overall transaction flow.

Shift Register: Handles the **bit-level serial shifting** of data, managing both transmission and reception of data bits across the SPI interface.

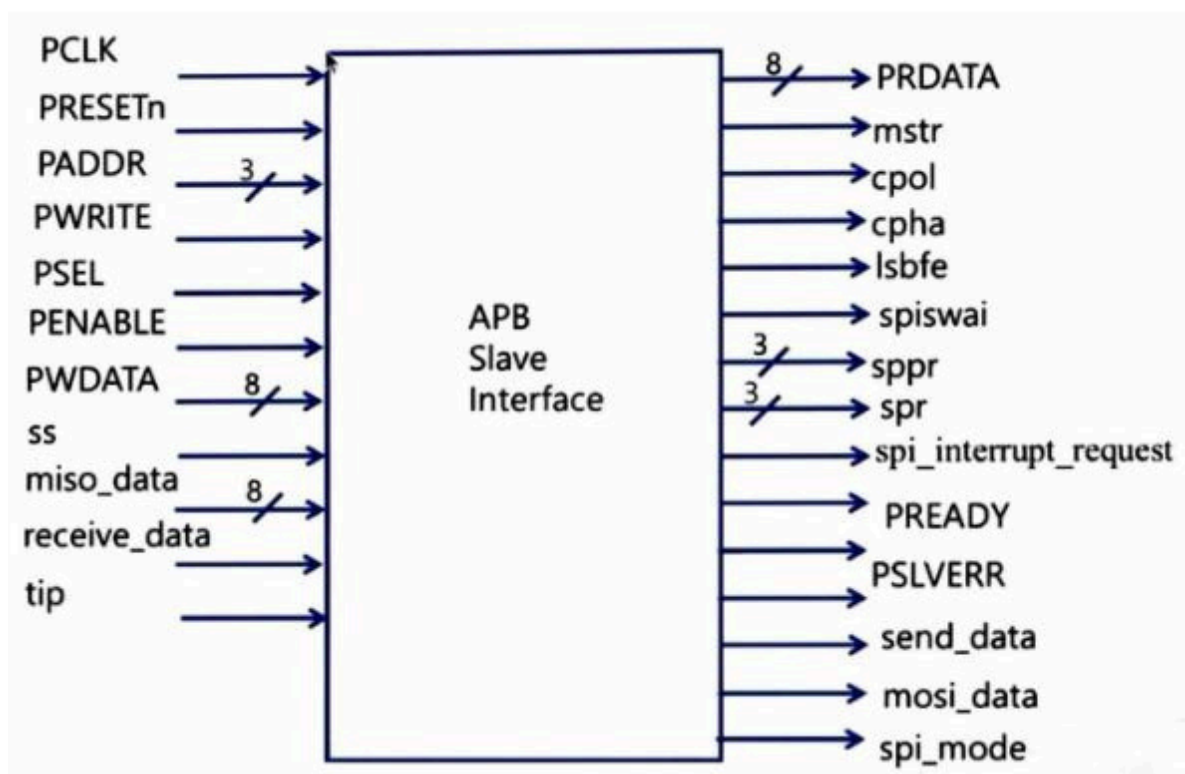
3.TOP Level Block Diagram:



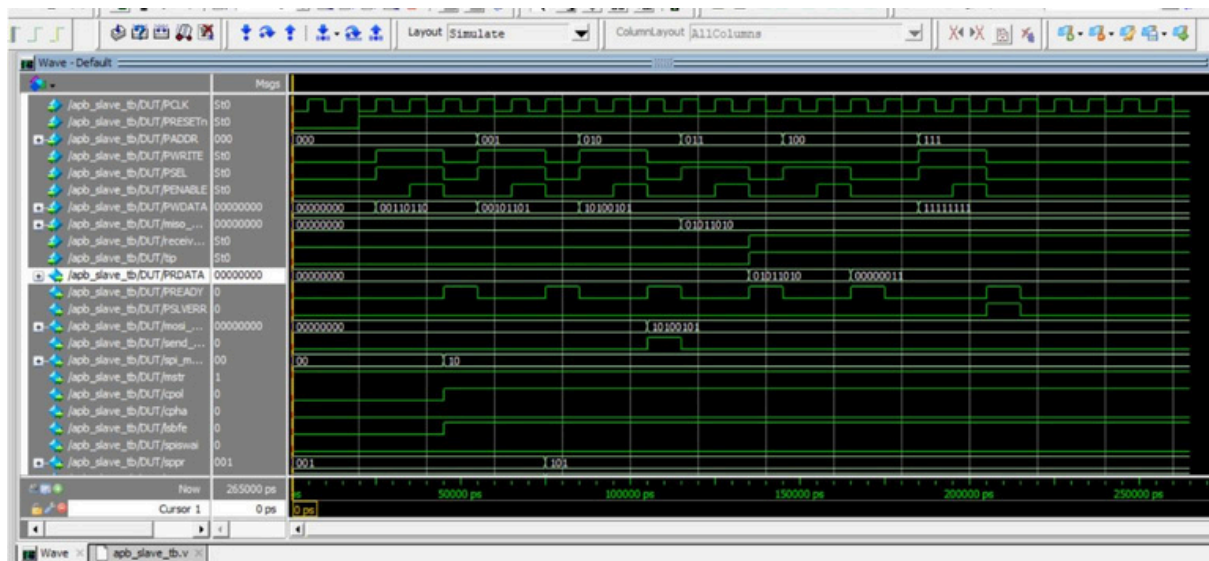
4. APB SLAVE INTERFACE:

APB slave interface sits at the top of the hierarchy, directly interfacing with the processor bus. It decodes **PADDR**, **PWDATA**, **PRDATA**, **PSEL**, **PENABLE**, and **PWRITE** signals, maps them to internal configuration registers (mode, clock divider, data), and translates **APB** read/write transactions so the rest of the design has no knowledge of the bus protocol.

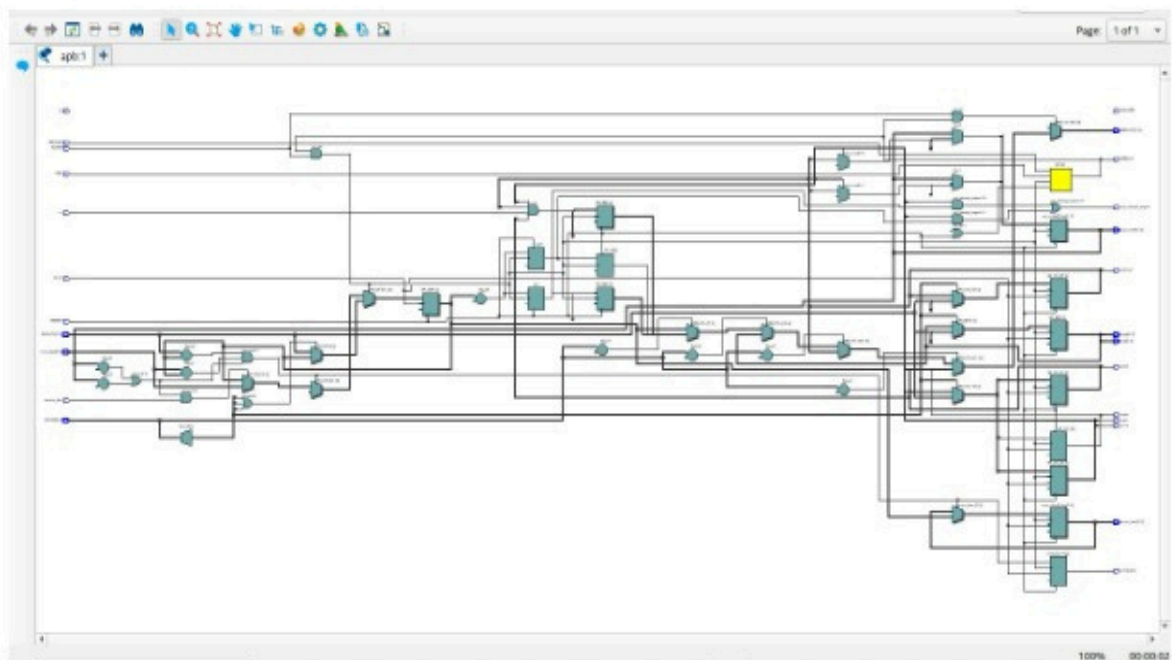
4.1 Block Diagram:



4.2 Simulation Waveform:



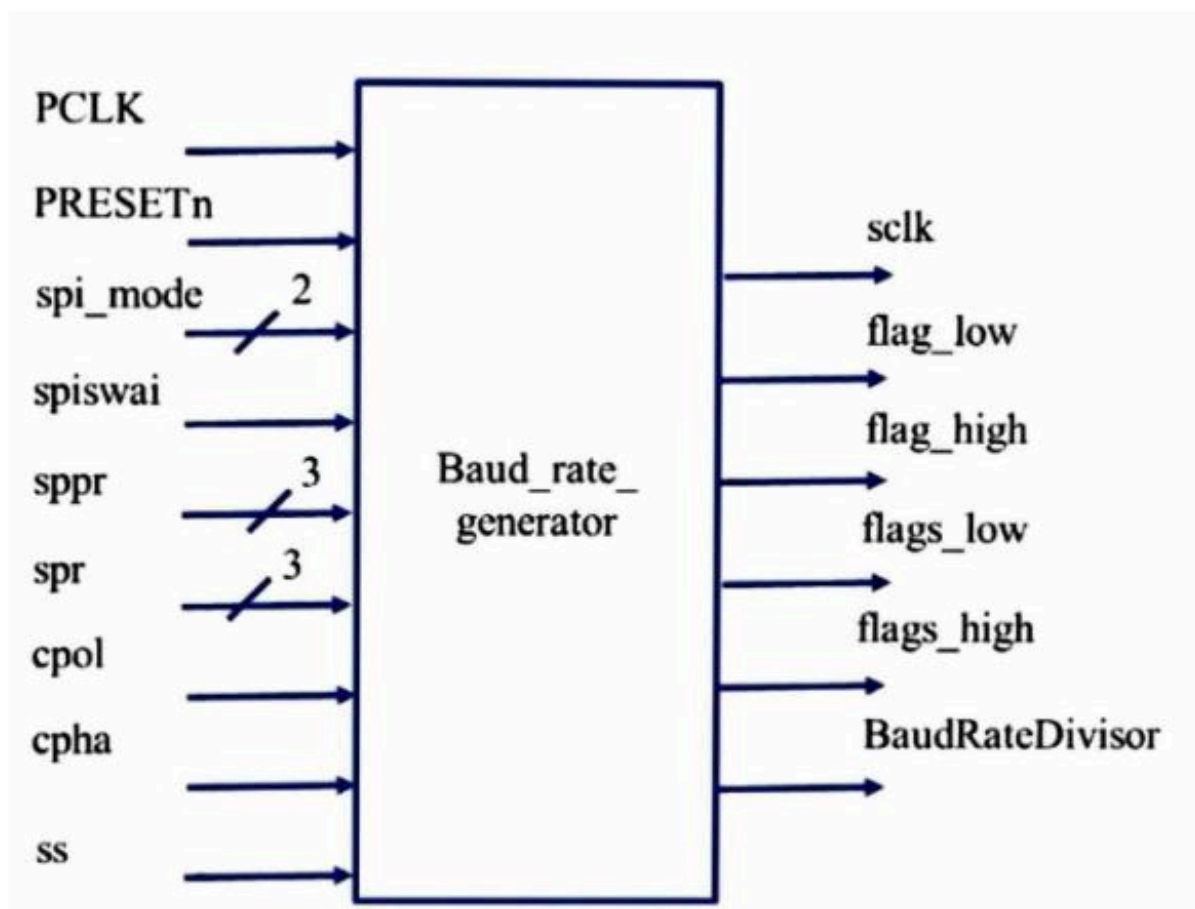
4.3 Synthesis results:



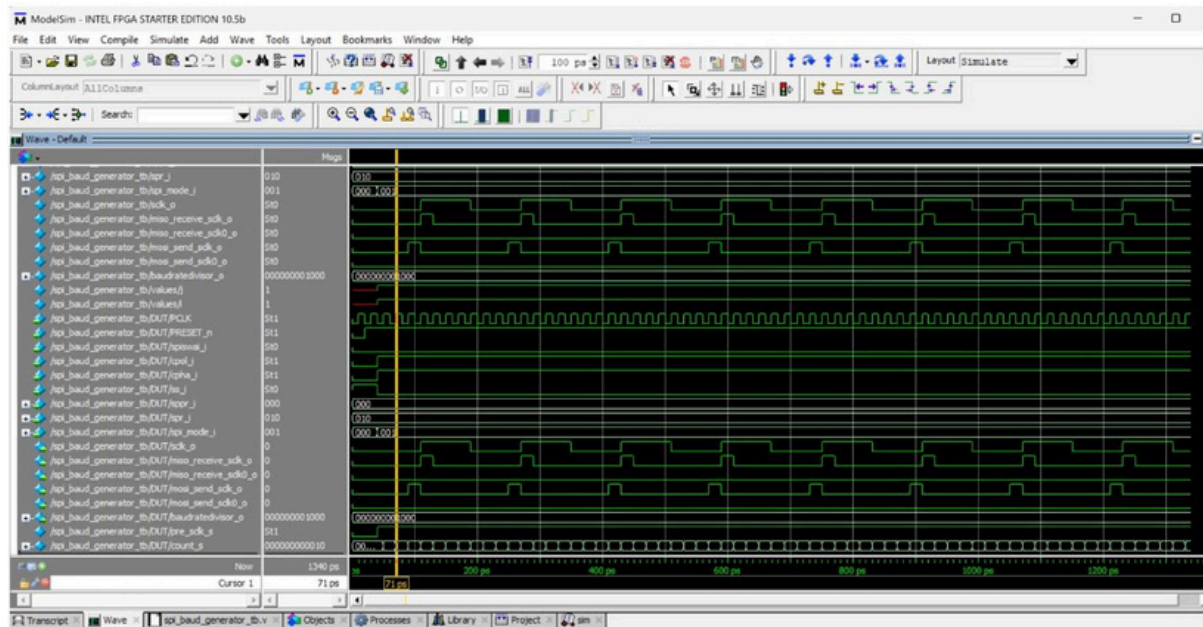
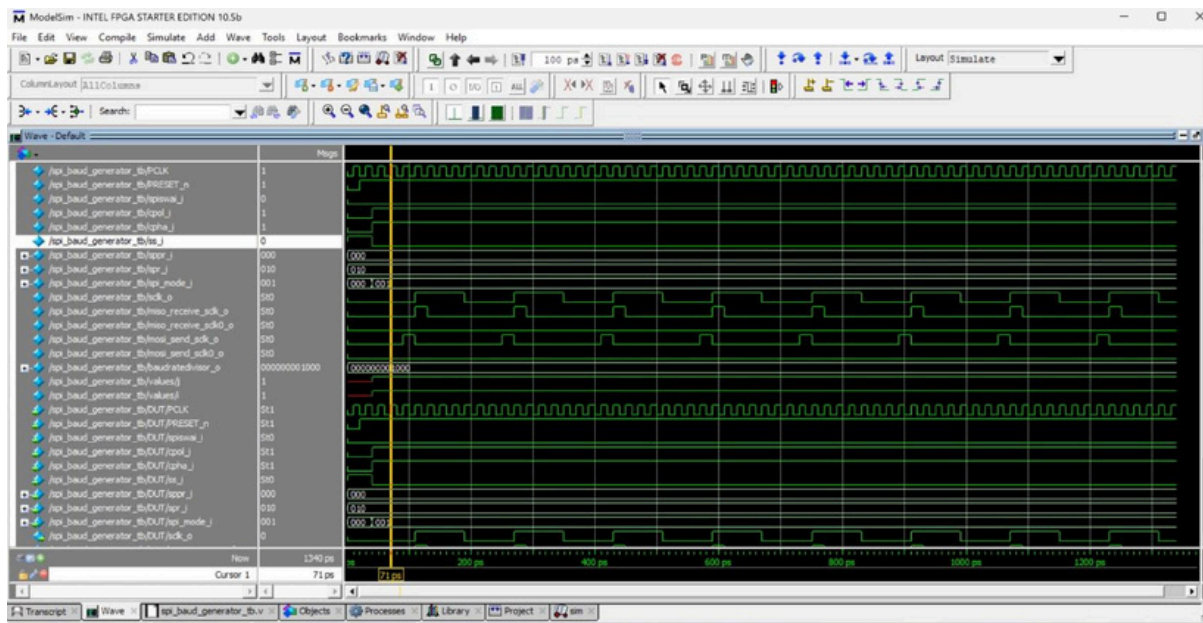
5.BAUD RATE GENERATOR:

Baud rate generator takes PCLK and a clock-divider value written by software through the **APB slave interface**, then produces a SCLK tick at the desired SPI bit rate. The divider ratio ($PCLK / (2 \times \text{divider})$) determines SPI speed. This block keeps the timing source cleanly separated so the shift register and control logic can simply consume a strobe.

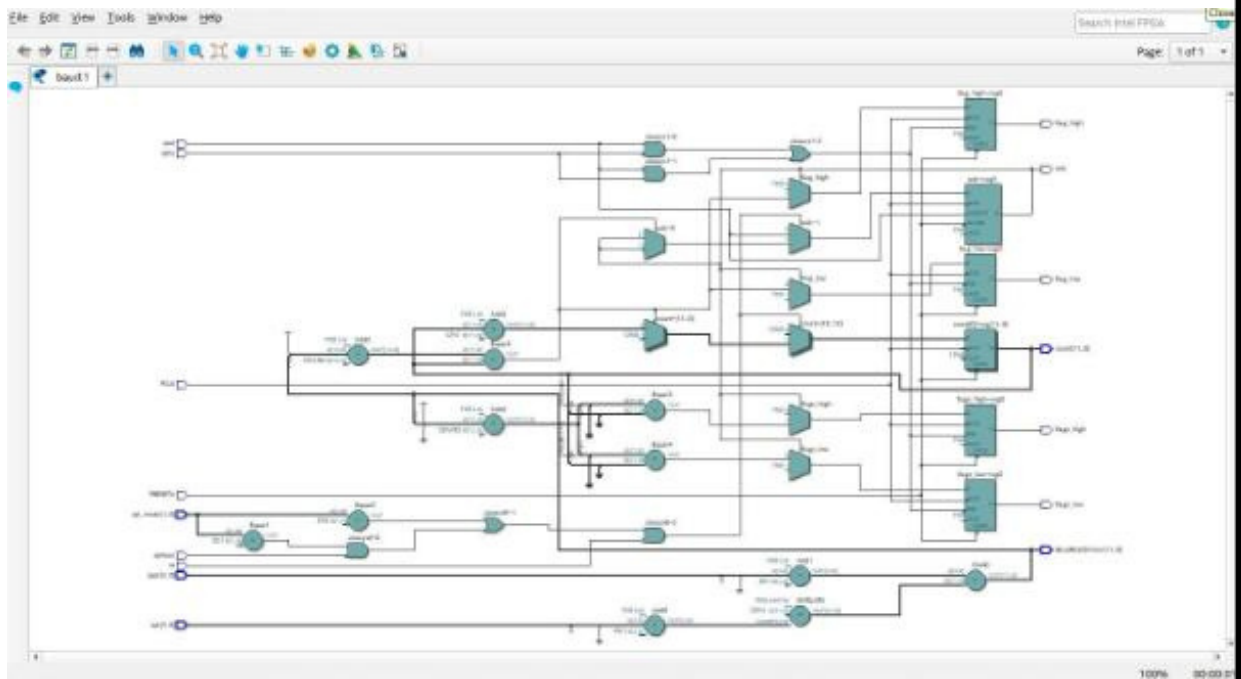
5.1 Block Diagram:



5.2 Simulation Waveforms:



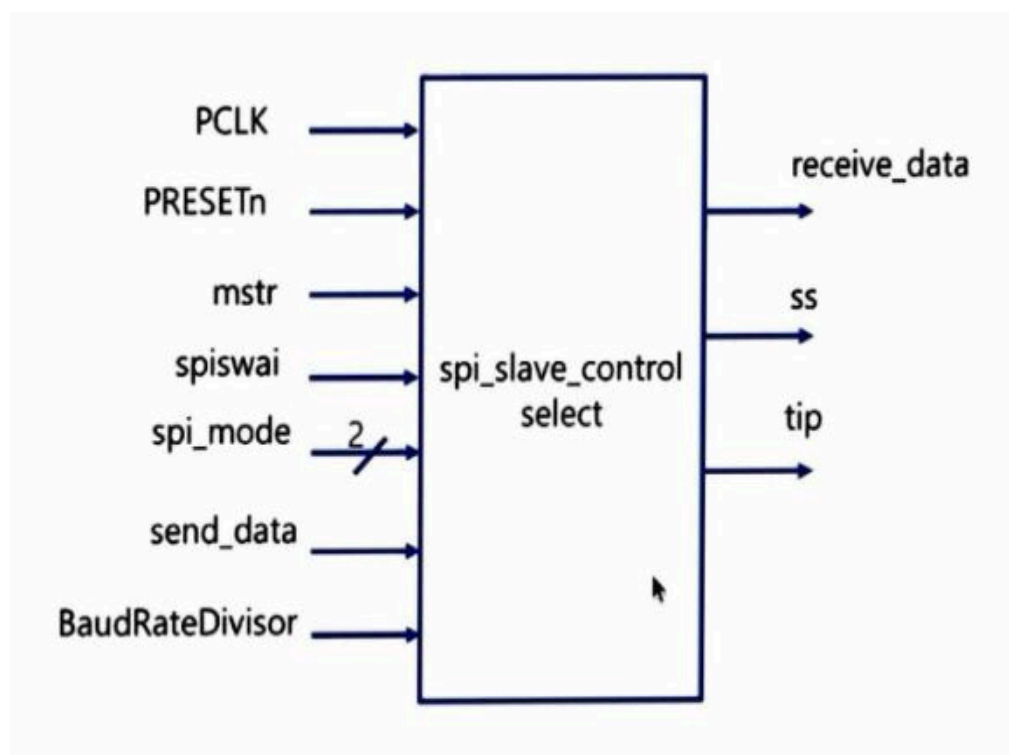
5.3 Synthesis results:

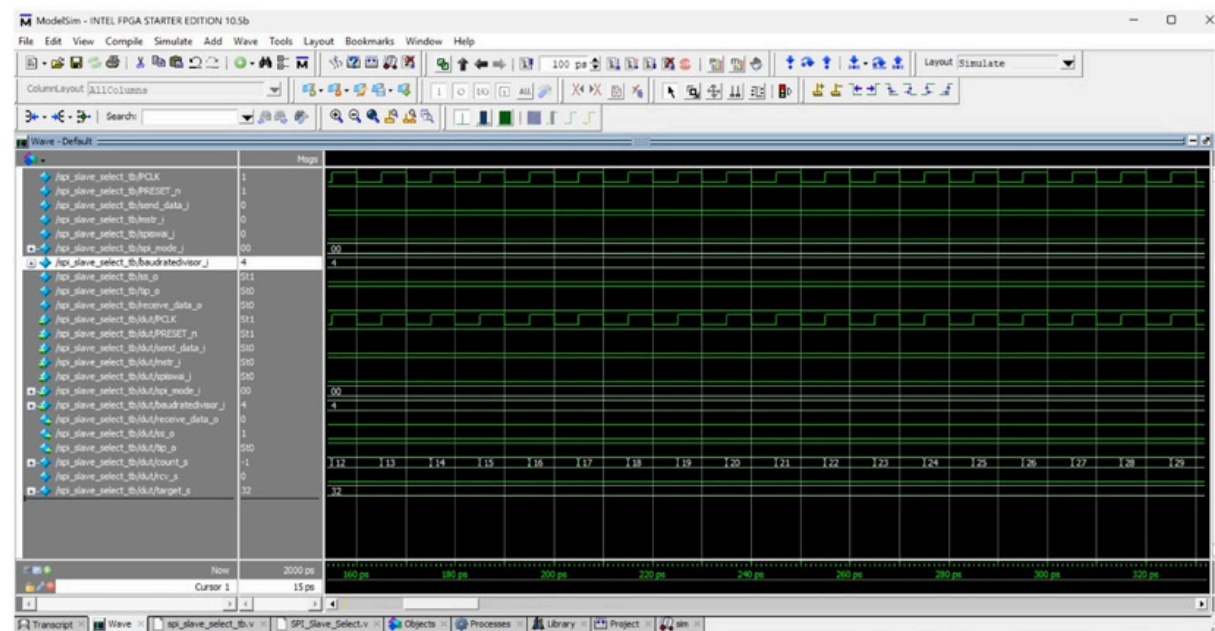


6.SPI SLAVE CONTROL SELECT:

SPI slave control & select manages the chip-select ($\overline{CS}$) line, SPI mode (CPOL/CPHA polarity and phase), and the enable/idle/transfer state machine. It interprets the configuration registers to assert $\overline{CS}$ to the correct external device, arms the shift register at the start of a frame, and flags transfer-complete status back through the **APB interface**.

6.1 Block Diagram:



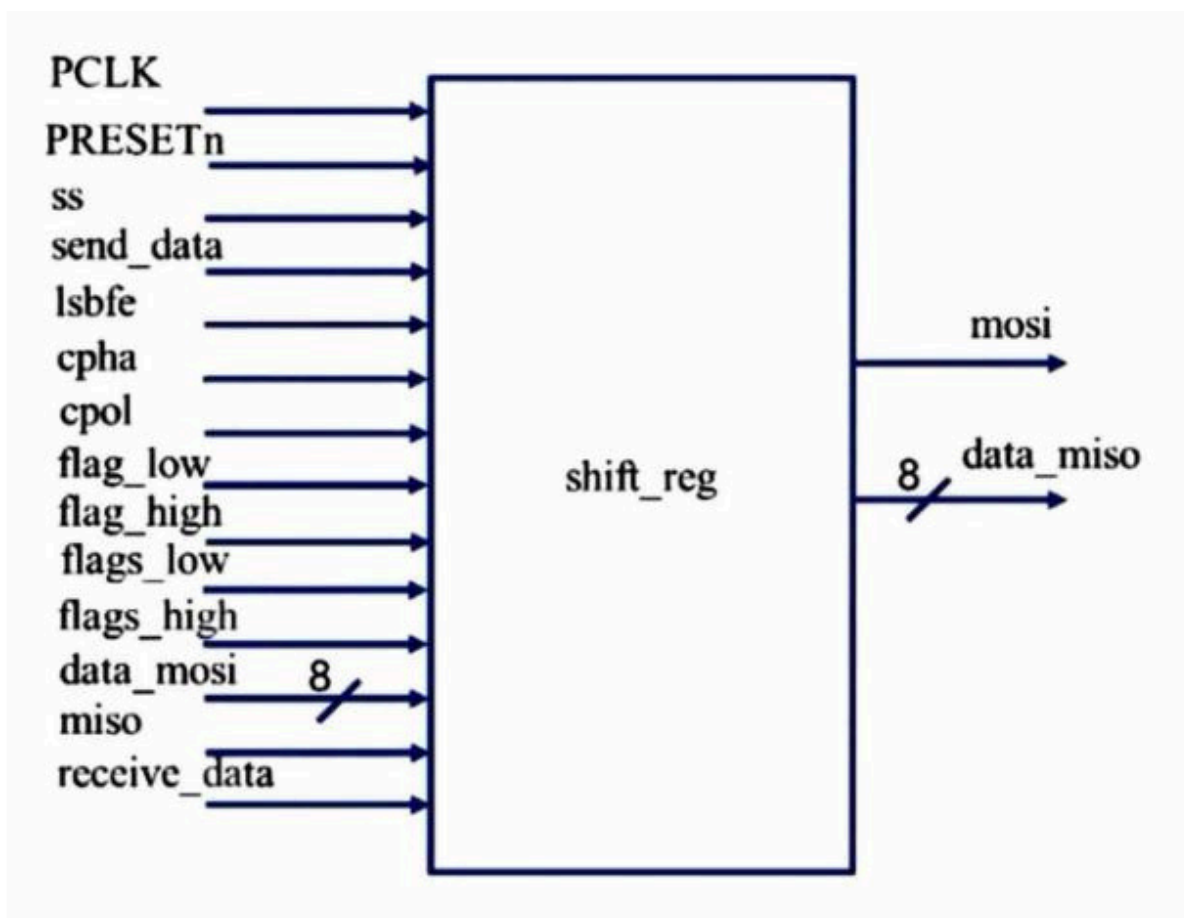
[illegible]



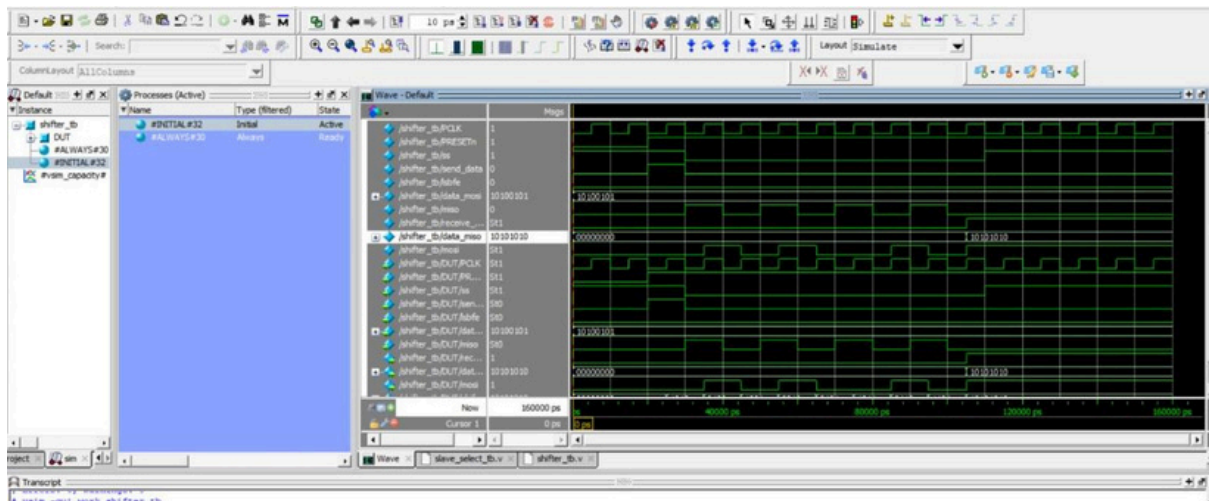
7. SHIFT REGISTER:

Shift register is the actual data path: it clocks **MOSI** bits in from the external device on each SCLK edge, simultaneously shifts **MISO** bits out, and when a full byte (or word) has been exchanged it raises a done flag so the **APB slave interface** can present the received data as a readable register.

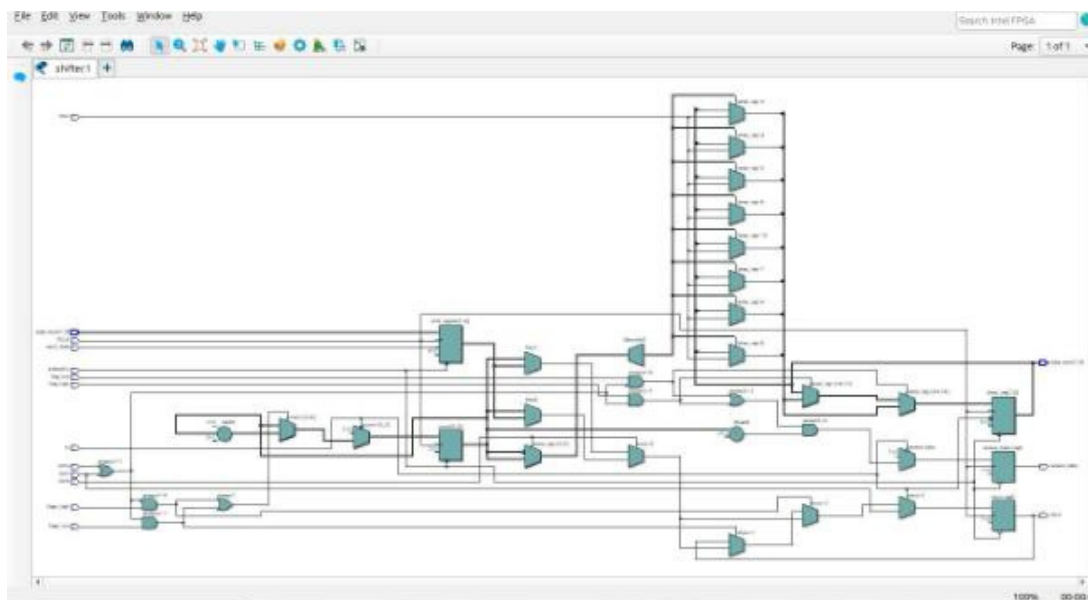
7.1 Block Diagram:



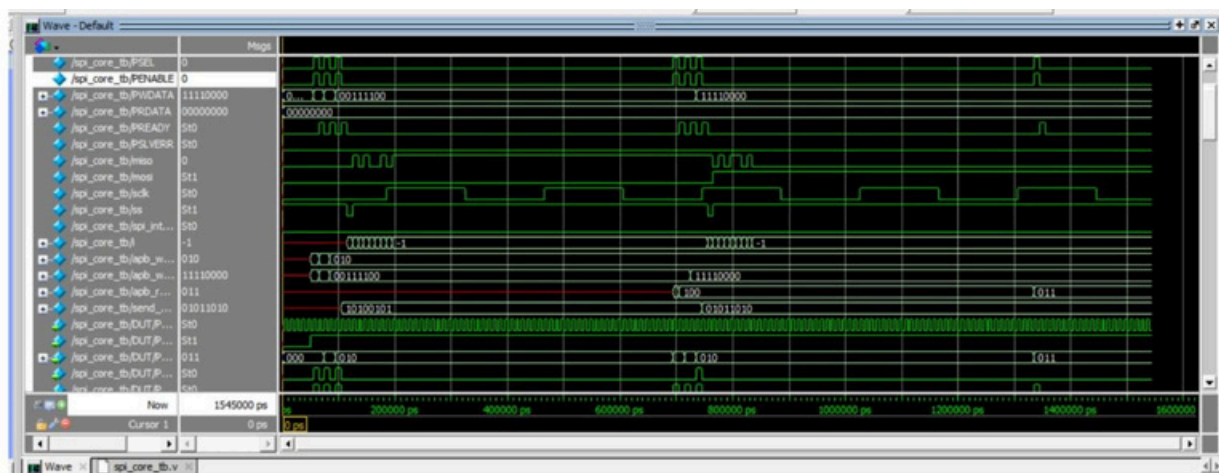
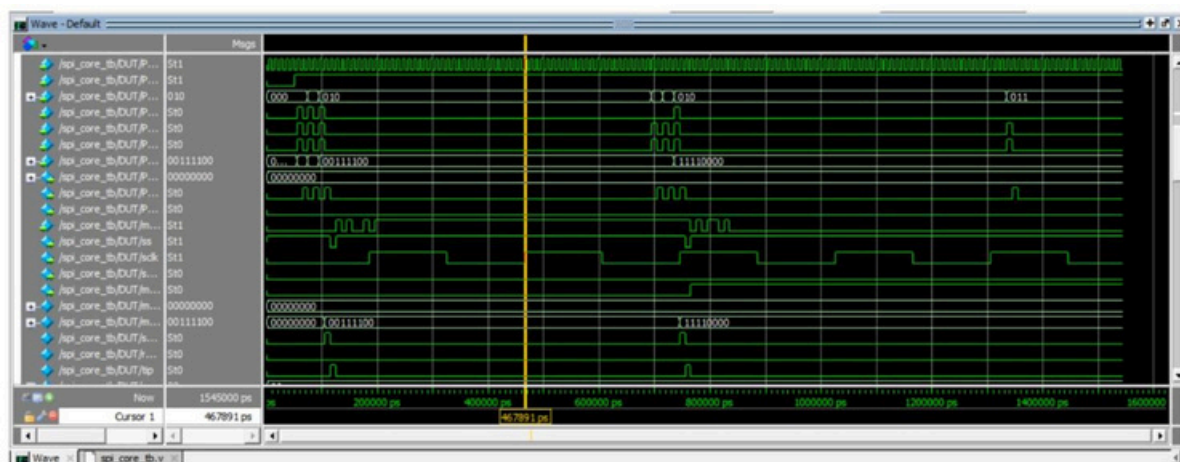
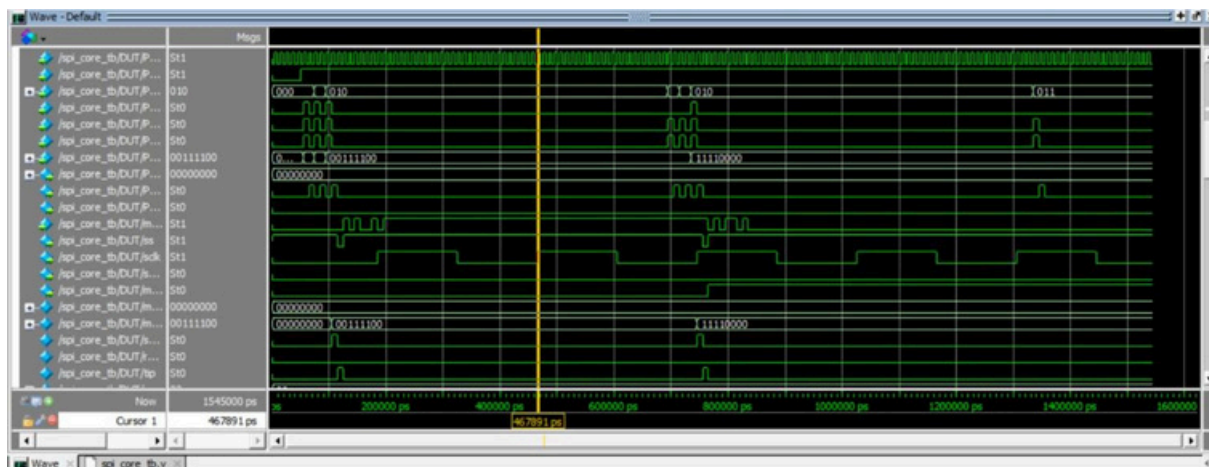
7.2 Simulation Waveform:



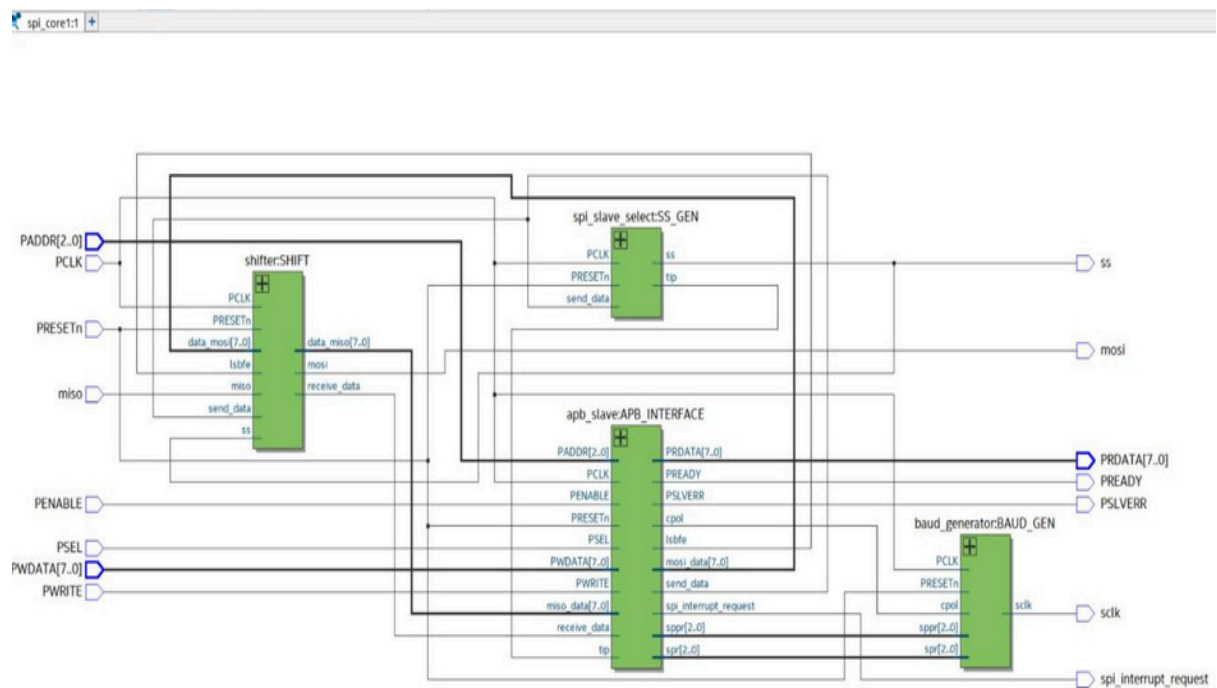
7.3 Synthesis results:



8.SPI CORE Waveforms:



9. Top Module Synthesis Result:



10.CONCLUSION:

The **APB-Based SPI Design** project was successfully realized, enabling efficient and reliable communication between an **AMBA APB bus** and **SPI-compatible peripheral devices**. By integrating the **APB interface** with **SPI control logic**, the design ensures dependable data transfer while offering straightforward configurability through dedicated **control** and **status registers**.

The **modular architecture** of the design enhances scalability, streamlines the verification process, and supports seamless integration into **SoC-based systems**. Simulation results confirmed the correct functional behavior of both the **APB** and **SPI protocols**, validating successful **data transmission and reception** across all tested scenarios.

Looking ahead, potential future enhancements may include support for **multiple SPI slaves**, **DMA-based data transfer**, and **FPGA implementation** for comprehensive performance evaluation. Overall, this project delivers a robust, reusable, and efficient **SPI communication solution** well-suited for modern **embedded systems** and **next-generation SoC designs**.